BUS COMMUNICATION PROTOCOL VERIFICATION - AMBA AXI BUS PROTOCOL

DARIO FRIŠKAN
UNIVERSITATEA POLITEHNICA TIMIȘOARA
AUTOMATICĂ ȘI CALCULATOARE – CTI_RO

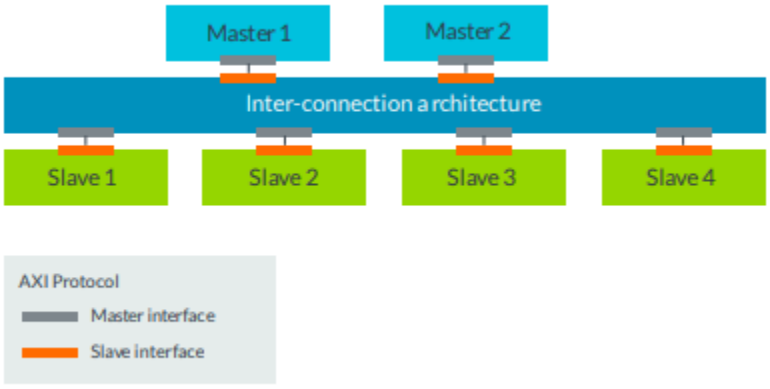
1. Project overview

The goal of this project is to design and verify the AMBA AXI (Advanced eXtensible Interface) bus communication protocol using SystemVerilog. The AMBA AXI protocol is widely used in modern system-on-chip (SoC) designs due to its ability to support high-speed communication between multiple masters and slaves. In this project, the focus is on verifying key features of the AXI protocol, including data transfer, address decoding, burst transactions, and handling of multiple outstanding transactions.

To achieve this, we will create a SystemVerilog-based design of the AMBA AXI bus, implement various verification strategies, and validate the functionality of the design using both directed and random test cases. Additionally, we will extend the design to handle out-of-order transactions and ensure that the protocol manages varying transaction priorities correctly.

2. Design

2.1 AMBA AXI Bus Protocol Overview



AMBA AXI is a high-performance bus protocol that provides efficient and flexible communication in SoC designs. Key features of the AXI protocol include:

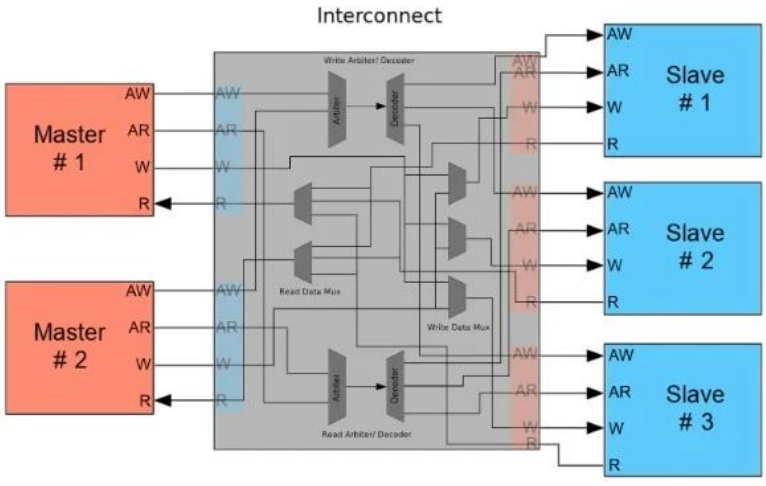
- **Multiple Masters and Slaves**: The AXI bus supports multiple masters that can initiate transactions and multiple slaves that respond to those transactions.

- **Burst Transfers**: AXI supports burst transactions, including fixed, incrementing, and wrapping bursts, to optimize data transfers.
- **Out-of-Order Transactions**: AXI allows for out-of-order transactions, enabling more efficient transaction scheduling.
- **Multiple Outstanding Transactions**: The protocol can handle multiple requests from a master simultaneously.

2.2 Design Components

The design consists of the following main components:

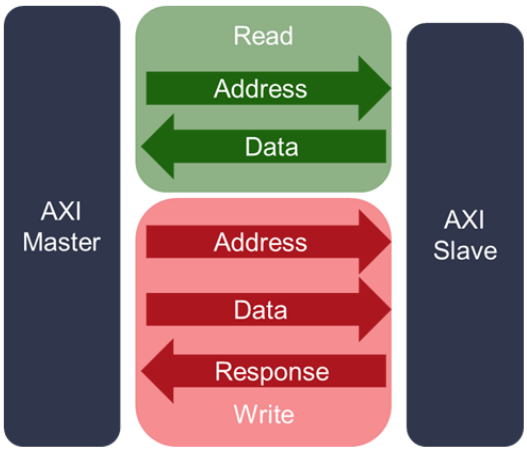
- **AXI Master**: Initiates read and write transactions on the AXI bus.
- **AXI Slave**: Responds to the requests from the AXI master, sending read data or acknowledgments.
- **AXI Interconnect**: The interconnect connects multiple masters and slaves and routes the requests based on address decoding.
- **AXI Bus**: The physical bus that carries data, address, and control signals between the master, interconnect, and slave.



2.3 High-Level Architecture

The high-level architecture of the design is as follows:

- The **AXI Master** generates read and write requests, including address, data, and control signals.
- The **AXI Interconnect** decodes the address and routes the request to the appropriate **AXI Slave**.
- The **AXI Slave** responds with data for read requests or an acknowledgment for write requests.
- **Burst Handling**: The interconnect ensures proper handling of burst transfers, including incrementing and wrapping bursts.



AXI Read and Write Channels

3. Verification plan

3.1 Verification objectives

The primary objective of the verification is to ensure that the design adheres to the AMBA AXI protocol specification and functions correctly in various scenarios. The verification focuses on the following areas:

- **Address Decoding**: Ensure that the correct slave responds based on the address provided by the master.
- **Burst Transactions**: Verify that burst transactions are handled correctly, including fixed, incrementing, and wrapping bursts.

- **Multiple Outstanding Transactions**: Ensure the system can handle multiple outstanding requests from the master.
- **Out-of-Order Transactions**: Verify the proper handling of out-of-order transactions and transactions with varying priorities.
- **Error Handling**: Test error conditions such as invalid addresses and slave readiness failures.

3.2 Verification Scenarios

The verification plan includes the following test scenarios:

Normal Transaction: A single read or write operation between a master and a slave.

- **Burst Transaction**: A series of transactions that constitute a burst transfer, using different burst types (fixed, incrementing, and wrapping).
- **Multiple Outstanding Transactions**: The master sends multiple requests, and the interconnect routes them properly to different slaves.
- **Out-of-Order Transactions**: Multiple transactions with varying priorities, ensuring the AXI protocol handles them correctly.
- **Edge Cases**: Test the system under abnormal conditions such as invalid addresses, slave unavailability, or other edge cases.
- **Error Handling**: Ensure the system reacts appropriately to error conditions such as timeout or incorrect data.

3.3 Test Coverage

To ensure full coverage of the AXI protocol, we will use both **directed tests** and **random tests**.

- **Directed Tests**: These tests are written explicitly to verify specific functionality, such as address decoding and burst handling.
- **Random Tests**: Randomly generated test cases will explore edge conditions and ensure the robustness of the system.
- **Functional Coverage**: Functional coverage will be tracked to ensure all critical areas of the AXI protocol are tested, such as burst types, out-of-order transactions, and multiple outstanding transactions.

4. Testbench

4.1 Testbench Architecture

The testbench is responsible for simulating the AXI bus protocol, validating its functionality across different scenarios. The architecture includes:

- **AXI Master**: Generates read and write transactions, including burst transactions (fixed, incrementing, and wrapping), and handles out-of-order transactions. It manages signals like axi_arvalid, axi_awvalid, and axi_wvalid.
- **AXI Slave**: Responds to read and write requests, sending data or acknowledgment signals using axi_rvalid and axi_bvalid.
- **AXI Interconnect**: Routes the requests between multiple masters and slaves based on address decoding, ensuring that the correct slave responds.
- **Simulation Environment**: A control module that initializes the design, applies stimulus, and runs the simulation to validate the functionality.

4.2 Testbench Code

The testbench instantiates the **AXI Master**, **AXI Slave**, and the **AXI Interconnect**. It includes multiple test cases to cover normal operations like read/write transactions, burst transfers, and edge cases, including error handling (invalid addresses, slave readiness failures). Directed tests are written to validate specific functionality, and random tests ensure robustness.

```
module axi_tb;

  // Declare signals
  reg clk;
  reg reset;
  reg [31:0] addr;
  reg [31:0] wr_data;
  wire [31:0] rd_data;
  reg [3:0] axi_arvalid;
  reg [3:0] axi_awvalid;
  reg [3:0] axi_wvalid;
  wire [3:0] axi_rvalid;
  wire [3:0] axi_bvalid;
  reg [3:0] axi_rready;
  reg [3:0] axi_bready;

  // Instantiate master, slave, and interconnect
  axi_master master1 (
    .clk(clk),
    .reset(reset),
    .addr(addr),
    .wr_data(wr_data),
    .axi_arvalid(axi_arvalid),
    .axi_awvalid(axi_awvalid),
    .axi_wvalid(axi_wvalid),
    .axi_rready(axi_rready),
    .axi_bready(axi_bready)
  );

  axi_slave slave1 (
    .clk(clk),
    .reset(reset),
    .addr(addr),
    .wr_data(wr_data),
    .rd_data(rd_data),
    .axi_arvalid(axi_arvalid),
    .axi_awvalid(axi_awvalid),
    .axi_wvalid(axi_wvalid),
    .axi_rvalid(axi_rvalid),
    .axi_bvalid(axi_bvalid)
  );

  // Clock generation
  always #5 clk = ~clk;

  // Reset logic
  initial begin
    clk = 0;
    reset = 1;
    #10 reset = 0;
  end

  // Test sequence
  initial begin
    // Test 1: Single Read Transaction
    addr = 32'h0000_0000;
    wr_data = 32'hA5A5_A5A5;
    axi_arvalid = 4'b0001;
```

```
  // Reset logic
  initial begin
    clk = 0;
    reset = 1;
    #10 reset = 0;
  end

  // Test sequence
  initial begin
    // Test 1: Single Read Transaction
    addr = 32'h0000_0000;
    wr_data = 32'hA5A5_A5A5;
    axi_arvalid = 4'b0001;
    axi_awvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;
    axi_arvalid = 4'b0000;
    #10; // Wait for response

    // Test 2: Burst Write Transaction
    addr = 32'h1000_0000;
    wr_data = 32'h1234_5678;
    axi_awvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #20;
    axi_awvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

    // Test 3: Out-of-Order Transactions
    addr = 32'h2000_0000;
    wr_data = 32'hDEAD_BEEF;
    axi_awvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    addr = 32'h3000_0000;
    wr_data = 32'hBEEF_DEAD;
    axi_awvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    axi_awvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

    // Test 4: Error Handling (e.g., invalid address)
    addr = 32'hFF00_FFFF;
    wr_data = 32'h9999_9999;
    axi_awvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    axi_awvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

  end
endmodule
```

4.3 Master Module

The AXI Master generates read and write transactions, managing signals such as axi_arvalid, axi_awvalid, and axi_wvalid. It initiates transactions based on specific conditions and is synchronized with the clock and reset signals.

```
module axi_master(
  input clk,
  input reset,
  input [31:0] addr,
  input [31:0] wr_data,
  output reg [3:0] axi_arvalid,
  output reg [3:0] axi_awvalid,
  output reg [3:0] axi_wvalid,
  output reg axi_rready,
  output reg axi_bready
);
  // Internal state and transaction logic
  reg [31:0] data_buffer;

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      axi_arvalid <= 0;
      axi_awvalid <= 0;
      axi_wvalid <= 0;
      axi_rready <= 0;
      axi_bready <= 0;
      data_buffer <= 0;
    end else begin
      // Read transaction
      if (axi_arvalid == 4'b0000) begin
        axi_arvalid <= 4'b0001; // Initiate read
      end
      // Write transaction
      if (axi_awvalid == 4'b0000 && axi_wvalid == 4'b0000) begin
        axi_awvalid <= 4'b0001; // Initiate write
        axi_wvalid <= 4'b0001;
      end
    end
  end
endmodule
```

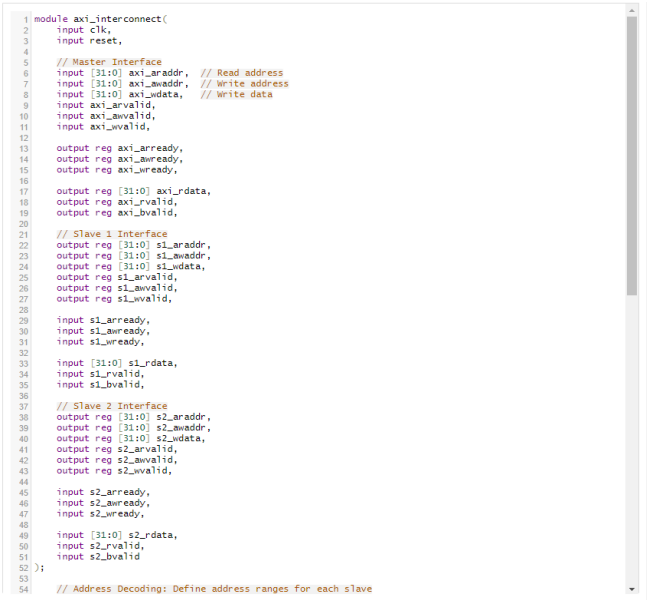
4.4 Slave Module

Responds to read and write requests from the master. It handles signals like axi_arvalid, axi_awvalid, and axi_wvalid, and sends back data or acknowledges writes using axi_rvalid and axi_bvalid.

```
module axi_slave(
  input clk,
  input reset,
  input [31:0] addr,
  input [31:0] wr_data,
  output reg [31:0] rd_data,
  input [3:0] axi_arvalid,
  input [3:0] axi_awvalid,
  input [3:0] axi_wvalid,
  output reg [3:0] axi_rvalid,
  output reg [3:0] axi_bvalid
);
  reg [31:0] memory [0:255]; // Memory simulation for slave
  reg [31:0] rd_data_buffer;

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      rd_data <= 0;
      axi_rvalid <= 0;
      axi_bvalid <= 0;
    end else begin
      // Handle read
      if (axi_arvalid) begin
        rd_data <= memory[addr[7:0]]; // Read from memory
        axi_rvalid <= 4'b0001; // Assert read valid
      end
      // Handle write
      if (axi_awvalid && axi_wvalid) begin
        memory[addr[7:0]] <= wr_data; // Write to memory
        axi_bvalid <= 4'b0001; // Assert write valid
      end
    end
  end
endmodule
```

4.5 Interconnect Module

The AXI Interconnect is responsible for routing transactions between the AXI Master and multiple AXI Slaves based on address decoding. It ensures that read and write requests are directed to the correct slave, allowing efficient communication in a multi-slave system.



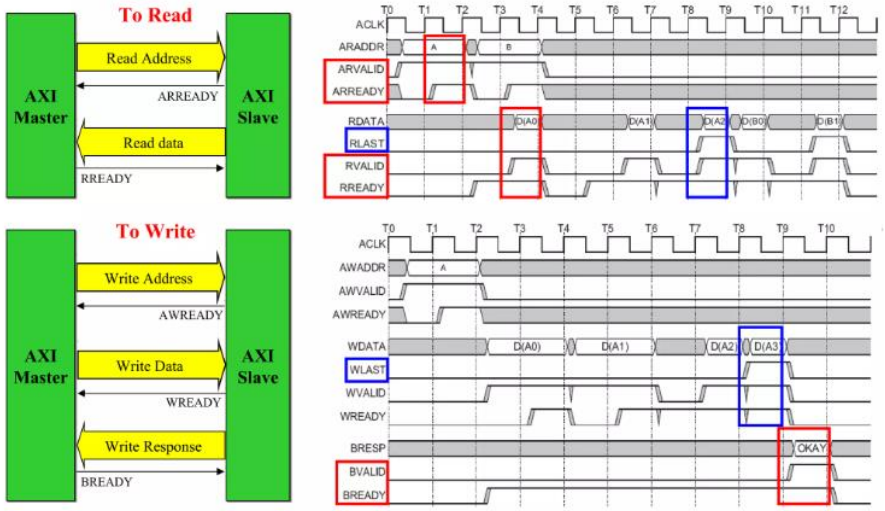
```
module axi_interconnect(
    input clk,
    input reset,

    // Master Interface
    input [31:0] axi_araddr,  // Read address
    input [31:0] axi_awaddr,  // Write address
    input [31:0] axi_wdata,   // Write data
    input axi_arvalid,
    input axi_awvalid,
    input axi_wvalid,

    output reg axi_arready,
    output reg axi_awready,
    output reg axi_wready,

    output reg [31:0] axi_rdata,
    output reg axi_rvalid,
    output reg axi_bvalid,

    // Slave 1 Interface
    output reg [31:0] s1_araddr,
    output reg [31:0] s1_awaddr,
    output reg [31:0] s1_wdata,
    output reg s1_arvalid,
    output reg s1_awvalid,
    output reg s1_wvalid,

    input s1_arready,
    input s1_awready,
    input s1_wready,

    input [31:0] s1_rdata,
    input s1_rvalid,
    input s1_bvalid,

    // Slave 2 Interface
    output reg [31:0] s2_araddr,
    output reg [31:0] s2_awaddr,
    output reg [31:0] s2_wdata,
    output reg s2_arvalid,
    output reg s2_awvalid,
    output reg s2_wvalid,

    input s2_arready,
    input s2_awready,
    input s2_wready,

    input [31:0] s2_rdata,
    input s2_rvalid,
    input s2_bvalid
);

    // Address Decoding: Define address ranges for each slave
    localparam S1_BASE_ADDR = 32'h0000_0000;
    localparam S2_BASE_ADDR = 32'h1000_0000;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            axi_arready <= 0;
            axi_awready <= 0;
            axi_wready <= 0;
            axi_rvalid <= 0;
            axi_bvalid <= 0;
            axi_rdata <= 32'b0;

            s1_arvalid <= 0;
            s1_awvalid <= 0;
            s1_wvalid <= 0;

            s2_arvalid <= 0;
            s2_awvalid <= 0;
            s2_wvalid <= 0;
        end else begin
            // Address decoding for READ transactions
            if (axi_arvalid) begin
                if (axi_araddr >= S1_BASE_ADDR && axi_araddr < S2_BASE_ADDR) begin
                    s1_arvalid <= 1;
                    s1_araddr <= axi_araddr;
                end else if (axi_araddr >= S2_BASE_ADDR) begin
                    s2_arvalid <= 1;
                    s2_araddr <= axi_araddr;
                end
            end

            // Address decoding for WRITE transactions
            if (axi_awvalid) begin
                if (axi_awaddr >= S1_BASE_ADDR && axi_awaddr < S2_BASE_ADDR) begin
                    s1_awvalid <= 1;
                    s1_awaddr <= axi_awaddr;
                    s1_wvalid <= axi_wvalid;
                    s1_wdata <= axi_wdata;
                end else if (axi_awaddr >= S2_BASE_ADDR) begin
                    s2_awvalid <= 1;
                    s2_awaddr <= axi_awaddr;
                    s2_wvalid <= axi_wvalid;
                    s2_wdata <= axi_wdata;
                end
            end

            // Pass responses from slaves back to master
            axi_rvalid <= s1_rvalid | s2_rvalid;
            axi_rdata <= s1_rvalid ? s1_rdata : s2_rdata;
            axi_bvalid <= s1_bvalid | s2_bvalid;
        end
    end

endmodule
```

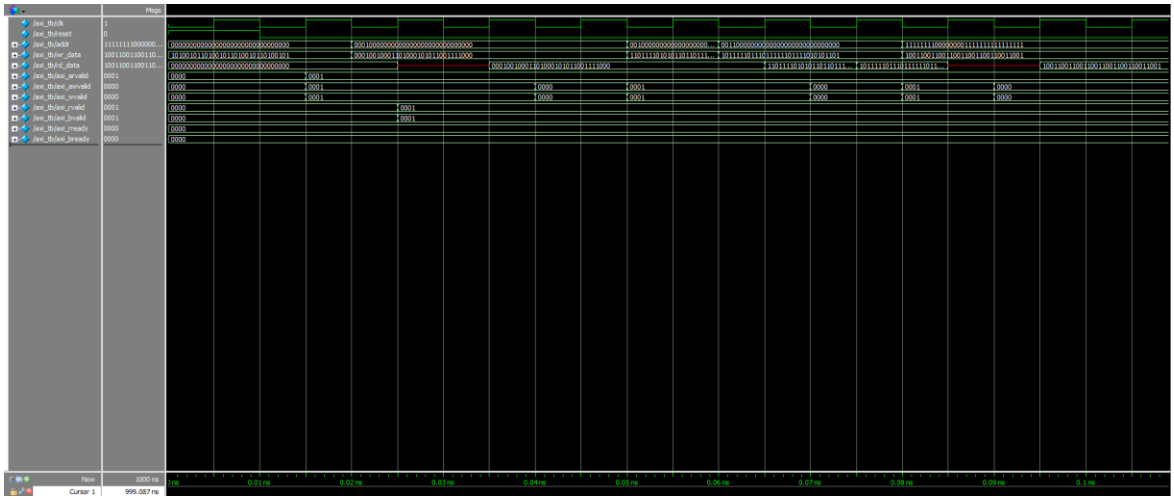
5. Simulation and Debugging

5.1 Waveform Analysis

The simulation results were analyzed using waveform viewers to verify that the system was correctly processing transactions. The following were checked:

- **Address Decoding**: Ensure that the correct slave was selected for each transaction based on the address.
- **Burst Handling**: Verify that burst transactions were correctly issued and handled.
- **Multiple Outstanding Transactions**: Ensure that multiple requests were properly managed and responded to.
- **Out-of-Order Transactions**: Check that transactions with varying priorities were handled in the correct order.





6.

6.1 Design Summary

The design consists of an AXI Master, AXI Slave, AXI Interconnect, and the AXI Bus. The AXI Master initiates read and write operations, and the AXI Slave responds. The AXI Interconnect handles address routing and burst transactions.

6.2 Testbench Summary

The testbench includes directed tests to verify normal read and write operations, as well as burst write transactions. Functional coverage analysis ensures that all fundamental aspects of the AXI protocol, such as address decoding, burst transactions, and multiple outstanding transactions, are thoroughly validated.

References:

1. https://www.allaboutcircuits.com/technical-articles/what-are-axi-interconnects-tutorial-master-slave-digital-logic/

2. Introduction to AMBA AXI4

3. AMBA® AXI Protocol Specification

4. https://www.cis.upenn.edu/~cis5710/current/slides/13_axi.pdf

5. https://adaptivesupport.amd.com/s/article/1053914?language=en_US

6. https://habr.com/en/articles/572926/

7. https://www.slideshare.net/AzadMishra1/axi-protocol-55779579